```
import { randomBytes, randomUUID } from 'node:crypto';
import { tx } from './db.js';
import { MAX_CARTONES_POR_COMPRA, PRECIO_CARTON, PRECIO_ENTRADA, TOTAL_CARTONES }

export const TOTAL_NUMEROS = TOTAL_CARTONES;   // los "números" son los cartones
export const RESERVA_MIN = 10;
const MAX_POR_ORDEN = MAX_CARTONES_POR_COMPRA;

export class ErrorNegocio extends Error {
  constructor(codigo, mensaje, extra = {}) {
    super(mensaje);
    this.codigo = codigo;
    this.extra = extra;
  }
}

const ALFABETO = 'ABCDEFGHJKLMNPQRSTUVWXYZ23456789';
function nuevaRef() {
  let s = '';
  for (const b of randomBytes(8)) s += ALFABETO[b % ALFABETO.length];
  return `ORD-${s}`;
}


export const normalizarNombre = (v) => String(v ?? '').trim().replace(/\s+/g, ' '
// "mateo GÓMEZ" -> "Mateo Gómez" (las partículas de, del, la… quedan en minúscul
const PARTICULAS = new Set(['de', 'del', 'la', 'las', 'los', 'y', 'da', 'di', 'va
export const formatearNombre = (v) => normalizarNombre(v).split(' ').map((p, i) =
  const m = p.toLowerCase();
  return i > 0 && PARTICULAS.has(m) ? m : m.charAt(0).toUpperCase() + m.slice(1);
}).join(' ');
const PALABRA_NOMBRE = /^[A-Za-zÁÉÍÓÚÜÑáéíóúüñ][A-Za-zÁÉÍÓÚÜÑáéíóúüñ'.-]*$/;
// Nombre completo: al menos dos palabras de 2 letras o más (nombre y apellido).
export function nombreCompletoValido(v) {
  const n = normalizarNombre(v);
  if (!n || n.length > 80) return false;
  const palabras = n.split(' ');
  return palabras.length >= 2 && palabras.every(
    (p) => PALABRA_NOMBRE.test(p) && p.replace(/[^A-Za-zÁÉÍÓÚÜÑáéíóúüñ]/g, '').le
}

// Números libres: sin dueño, o reservados con la reserva vencida.
const DISPONIBLE = `(estado = 'libre' or (estado = 'reservado' and reservado_hast
```

```
// Toma n números al azar bloqueando las filas; las que otra transacción ya
// tiene bloqueadas se saltean (no hay esperas ni deadlocks).
async function tomarAzar(c, n, excluir = []) {
  const tomados = [];
  for (let intento = 0; tomados.length < n && intento < 6; intento++) {
    const r = await c.query(
      `select numero from numeros where ${DISPONIBLE} and numero <> all($2::small
       order by random() limit $1 for update skip locked`,
      [n - tomados.length, [...excluir, ...tomados]],
    );
    tomados.push(...r.rows.map((x) => x.numero));
  }
  return tomados;
}

// Bloquea, en orden ascendente (evita deadlocks), los números pedidos que sigan
async function tomarElegidos(c, lista) {
  const r = await c.query(
    `select numero from numeros where numero = any($1::smallint[]) and ${DISPONIB
     order by numero for update`,
    [lista],
  );
  return r.rows.map((x) => x.numero);
}

/**
 * Devuelve el celular en formato internacional de Argentina (549 + área + número
 * puede ser un celular argentino: código de área + número, sin 0 ni 15, 10 dígit
 * Misma regla que usa la web (waNumero), para que ambos lados coincidan.
 */
export function normalizarWhatsapp(raw) {
  let d = String(raw ?? '').replace(/\D/g, '');
  if (d.startsWith('00')) d = d.slice(2);
  if (d.startsWith('54') && d.length >= 12) { d = d.slice(2); if (d.startsWith('9
  d = d.replace(/^0+/, '');
  if (d.length === 12) {   // código de área + 15 + número (formato viejo)
    for (let a = 2; a <= 4; a++) { if (d.substr(a, 2) === '15') { d = d.slice(0,
  }
  if (d.length !== 10 || /^(\d)\1{9}$/.test(d) || d.startsWith('0')) return null;
  return `549${d}`;
}

function validarEntrada({ nombre, whatsapp, entradas, sueltos, modo, numeros }) {
  if (!String(nombre ?? '').trim() || !String(whatsapp ?? '').trim()) {
    throw new ErrorNegocio('datos_incompletos', 'Falta el nombre o el teléfono.')
  }
  if (!nombreCompletoValido(nombre)) {
```

```
    throw new ErrorNegocio('nombre_invalido', 'Escribí tu nombre y apellido.');
  }
  if (!normalizarWhatsapp(whatsapp)) {
    throw new ErrorNegocio('whatsapp_invalido',
      'Revisá tu teléfono: código de área y número, sin 0 ni 15. Por ejemplo: 11
  }
  if (!Number.isInteger(entradas) || !Number.isInteger(sueltos) || entradas < 0 |
    throw new ErrorNegocio('cantidad_invalida', 'Revisá la cantidad de entradas y
  }
  const cantidad = entradas + sueltos;
  if (cantidad < 1 || cantidad > MAX_POR_ORDEN) {
    throw new ErrorNegocio('cantidad_invalida', `Podés pedir de 1 a ${MAX_POR_ORD
  }
  if (modo !== 'azar' && modo !== 'elegir') throw new ErrorNegocio('modo_invalido
  if (modo === 'elegir') {
    if (!Array.isArray(numeros) || numeros.length !== cantidad) {
      throw new ErrorNegocio('numeros_invalidos', 'La cantidad de cartones elegid
    }
    const vistos = new Set();
    for (const n of numeros) {
      if (!Number.isInteger(n) || n < 1 || n > TOTAL_NUMEROS) {
        throw new ErrorNegocio('numeros_invalidos', `Los cartones van del 1 al ${
      }
      if (vistos.has(n)) {
        throw new ErrorNegocio('numeros_repetidos', `Repetiste el ${String(n).pad
      }
      vistos.add(n);
    }
  }
}

export const montoDe = (entradas, sueltos) => entradas * PRECIO_ENTRADA + sueltos

// Crea la orden y reserva los números (azar o elegidos) por RESERVA_MIN minutos.
export async function crearOrden(pool, entrada) {
  validarEntrada(entrada);
  const { nombre, whatsapp, entradas, sueltos, modo } = entrada;
  const cantidad = entradas + sueltos;
  const numeros = modo === 'elegir' ? [...entrada.numeros].sort((a, b) => a - b)

  return tx(pool, async (c) => {
    const id = randomUUID();
    const ref = nuevaRef();
    const monto = montoDe(entradas, sueltos);
    await c.query(
      `insert into ordenes (id, ref, comprador_nombre, comprador_whatsapp, cantid
                            numeros_elegidos, monto, expira_en)
```

```
       values ($1,$2,$3,$4,$5,$6,$7,$8,$9, now() + make_interval(mins => $10))`,
      [id, ref, formatearNombre(nombre), whatsapp.trim(), cantidad, entradas, mod
    );

    let reservados;
    if (modo === 'elegir') {
      reservados = await tomarElegidos(c, numeros);
      if (reservados.length !== cantidad) {
        const ok = new Set(reservados);
        const no = numeros.filter((n) => !ok.has(n)).map((n) => String(n).padStar
        throw new ErrorNegocio('numeros_no_disponibles', `No están disponibles: $
      }
    } else {
      reservados = await tomarAzar(c, cantidad);
      if (reservados.length !== cantidad) {
        throw new ErrorNegocio('sin_stock', 'No quedan cartones disponibles por a
      }
    }
    await c.query(
      `update numeros set estado = 'reservado', orden_id = $1,
              reservado_hasta = now() + make_interval(mins => $2)
       where numero = any($3::smallint[])`,
      [id, RESERVA_MIN, reservados],
    );
    const exp = await c.query('select expira_en from ordenes where id = $1', [id]
    return { id, ref, monto, expiraEn: exp.rows[0].expira_en };
  });
}

export async function guardarPreferencia(pool, ordenId, preferenciaId) {
  await pool.query('update ordenes set mp_preference_id = $2 where id = $1', [ord
}

// Si el pago no se pudo iniciar (falla de Mercado Pago), se libera la reserva.
export async function cancelarOrden(pool, ordenId) {
  await tx(pool, async (c) => {
    const o = await c.query('select estado from ordenes where id = $1 for update'
    if (o.rows[0]?.estado !== 'pendiente') return;
    await c.query(
      `update numeros set estado = 'libre', orden_id = null, reservado_hasta = nu
       where orden_id = $1 and estado = 'reservado'`, [ordenId]);
    await c.query(`update ordenes set estado = 'vencida' where id = $1`, [ordenId
  });
}

// Libera reservas vencidas. Se corre cada minuto.
export async function liberarVencidas(pool) {
```

```
  return tx(pool, async (c) => {
    const n = await c.query(
      `update numeros set estado = 'libre', orden_id = null, reservado_hasta = nu
       where estado = 'reservado' and reservado_hasta < now()`);
    const o = await c.query(
      `update ordenes set estado = 'vencida' where estado = 'pendiente' and expir
    return { numeros: n.rowCount, ordenes: o.rowCount };
  });
}

// ---------------------------------------------------------------------------
// Confirmación de pagos
// ---------------------------------------------------------------------------

const FINALES = ['asignado', 'requiere_reembolso', 'sin_orden', 'monto_distinto']

// Consigue los números de la orden al acreditarse el pago. Devuelve la lista o n
async function conseguirNumeros(c, ord) {
  const mios = await c.query(
    `select numero from numeros where orden_id = $1 and estado = 'reservado'
     order by numero for update`, [ord.id]);
  let tengo = mios.rows.map((r) => r.numero);

  if (ord.modo === 'elegir') {
    const faltan = ord.numeros_elegidos.filter((n) => !tengo.includes(n));
    if (faltan.length) {
      const re = await tomarElegidos(c, faltan);
      if (re.length !== faltan.length) return null;
      tengo = tengo.concat(re);
    }
  } else if (tengo.length < ord.cantidad) {
    const extra = await tomarAzar(c, ord.cantidad - tengo.length, tengo);
    if (tengo.length + extra.length < ord.cantidad) return null;
    tengo = tengo.concat(extra);
  }
  return tengo.sort((a, b) => a - b);
}

/**
 * Procesa el aviso de un pago de Mercado Pago. Es seguro llamarlo muchas veces,
 * y en paralelo, para el mismo pago:
 *  - el pago se consulta a la API de MP (no se confía en lo que llega en el avis
 *  - la orden se identifica SOLO por external_reference
 *  - un pago de MP (mp_payment_id) se registra una sola vez
 *  - una orden acepta como máximo un pago asignado
 */
export async function procesarPago(pool, mp, paymentId) {
```

```
const p = await mp.getPayment(paymentId);
const pid = String(p.id);

return tx(pool, async (c) => {
  // Serializa todo lo que toque este mismo pago.
  await c.query('select pg_advisory_xact_lock(hashtext($1))', [`pago:${pid}`]);

  const previo = (await c.query('select * from pagos where mp_payment_id = $1 f
  if (previo && FINALES.includes(previo.resultado)) {
    if (previo.estado_mp !== p.status) {
      await c.query(
        `update pagos set estado_mp = $2, actualizado_en = now(),
                detalle = coalesce(detalle,'') || ' | estado en MP cambió a ' |
         where mp_payment_id = $1`, [pid, p.status]);
    }
    return { resultado: previo.resultado, repetido: true };
  }

  const ord = p.external_reference
    ? (await c.query('select * from ordenes where ref = $1 for update', [p.exte
    : undefined;

  const guardar = (resultado, detalle = null) => c.query(
    `insert into pagos (mp_payment_id, orden_id, external_reference, estado_mp,
     values ($1,$2,$3,$4,$5,$6,$7)
     on conflict (mp_payment_id) do update
       set orden_id = excluded.orden_id, estado_mp = excluded.estado_mp, monto
           resultado = excluded.resultado, detalle = excluded.detalle, actualiz
    [pid, ord?.id ?? null, p.external_reference ?? null, p.status, Math.round(N

  if (p.status !== 'approved') {
    await guardar('no_aprobado', p.status);
    return { resultado: 'no_aprobado' };
  }
  if (!ord) {
    await guardar('sin_orden', 'external_reference desconocida');
    return { resultado: 'sin_orden' };
  }
  if (Number(p.amount) !== ord.monto) {
    await guardar('monto_distinto', `pagó ${p.amount}, la orden vale ${ord.mont
    return { resultado: 'monto_distinto' };
  }
  if (ord.estado === 'pagada' || ord.estado === 'requiere_reembolso' || ord.est
    await guardar('requiere_reembolso', `la orden ya estaba ${ord.estado}`);
    return { resultado: 'requiere_reembolso' };
  }
```

```
    const numeros = await conseguirNumeros(c, ord);
    if (!numeros) {
      await c.query(
        `update numeros set estado = 'libre', orden_id = null, reservado_hasta =
         where orden_id = $1 and estado = 'reservado'`, [ord.id]);
      await c.query(`update ordenes set estado = 'requiere_reembolso' where id =
      await guardar('requiere_reembolso', 'no quedaron números disponibles');
      return { resultado: 'requiere_reembolso' };
    }

    await c.query(
      `update numeros set estado = 'vendido', orden_id = $1, reservado_hasta = nu
       where numero = any($2::smallint[])`, [ord.id, numeros]);
    await c.query(`update ordenes set estado = 'pagada', pagada_en = now() where
    await guardar('asignado');

    await c.query(
      `insert into notificaciones (orden_id, tipo, payload) values ($1, 'comproba
      [ord.id, JSON.stringify({
        para: ord.comprador_whatsapp,
        comprador: ord.comprador_nombre,
        numeros: numeros.map((n) => String(n).padStart(4, '0')),
        entradas: ord.entradas,
      })]);
    return { resultado: 'asignado', numeros };
  });
}

// ---------------------------------------------------------------------------
// Consultas
// ---------------------------------------------------------------------------

export async function estadoOrden(pool, ref) {
  const o = (await pool.query('select * from ordenes where ref = $1', [ref])).row
  if (!o) return null;
  const out = { ref: o.ref, estado: o.estado, monto: o.monto, expiraEn: o.expira_
  if (o.estado === 'pagada') {
    out.comprador = o.comprador_nombre;
    const n = await pool.query(`select numero from numeros where orden_id = $1 an
    out.numeros = n.rows.map((r) => String(r.numero).padStart(4, '0'));
    out.entradas = o.entradas;
    out.sueltos = o.cantidad - o.entradas;
  }
  return out;
}

// Cartones que quedan (públicos: la web muestra "Quedan X de 500"). Cuenta libre
```

```
export async function stock(pool) {
  const r = await pool.query(`select count(*)::int as disponibles from numeros wh
  return { total: TOTAL_NUMEROS, disponibles: r.rows[0].disponibles };
}

export async function panel(pool) {
  const q = (sql, args) => pool.query(sql, args).then((r) => r.rows);
  const [tot] = await q(`select count(*)::int as vendidos from numeros where esta
  const [ven] = await q(
    `select count(*)::int as compras, coalesce(sum(monto), 0)::int as recaudado,
            coalesce(sum(entradas), 0)::int as entradas, coalesce(sum(cantidad -
     from ordenes where estado = 'pagada'`);
  // Registro de compras: pagadas y canceladas (estas últimas para devolver el di
  const compradores = await q(
    `select o.ref, o.comprador_nombre as comprador, o.comprador_whatsapp as whats
            o.entradas, (o.cantidad - o.entradas)::int as sueltos, o.cantidad,
            o.cortesia, o.monto_original, o.ajustada_en,
            coalesce(o.pagada_en, o.creada_en) as pagada_en, o.cancelada_en,
            (select p.mp_payment_id from pagos p where p.orden_id = o.id and p.re
            (select p.estado_mp from pagos p where p.orden_id = o.id and p.result
     from ordenes o
     where o.estado in ('pagada', 'cancelada')
     order by coalesce(o.cancelada_en, o.pagada_en, o.creada_en) desc`);
  return { vendidos: tot.vendidos, disponibles: TOTAL_NUMEROS - tot.vendidos, tot
}

/**
 * Cancela una compra desde el panel: libera sus números y la deja registrada com
 * Si ya tenía un pago aprobado, `devolver` trae los datos para devolverlo desde
 * (el sistema no mueve plata). Una compra cancelada no recibe números nunca más.
 */
export async function cancelarCompra(pool, ref) {
  return tx(pool, async (c) => {
    const o = (await c.query('select * from ordenes where ref = $1 for update', [
    if (!o) throw new ErrorNegocio('orden_inexistente', 'No encontramos esa compr
    const pago = (await c.query(
      `select mp_payment_id as id, estado_mp, monto from pagos where orden_id = $
    const devolver = pago && pago.estado_mp === 'approved' ? pago : null;
    if (o.estado === 'cancelada') return { ref: o.ref, estado: 'cancelada', repet
    const nums = (await c.query('select numero from numeros where orden_id = $1 o
    await c.query(
      `update numeros set estado = 'libre', orden_id = null, reservado_hasta = nu
    await c.query(
      `update ordenes set estado = 'cancelada', cancelada_en = now(), numeros_anu
    return { ref: o.ref, estado: 'cancelada', numeros: nums.map((n) => String(n).
  });
}
```

```
/** Borra por completo una compra YA cancelada (y sus avisos y pagos registrados)
export async function eliminarCompra(pool, ref) {
  return tx(pool, async (c) => {
    const o = (await c.query('select * from ordenes where ref = $1 for update', [
    if (!o) throw new ErrorNegocio('orden_inexistente', 'No encontramos esa compr
    if (o.estado !== 'cancelada') {
      throw new ErrorNegocio('primero_cancelar', 'Primero hay que cancelar la com
    }
    await c.query('delete from notificaciones where orden_id = $1', [o.id]);
    await c.query('delete from pagos where orden_id = $1', [o.id]);
    await c.query('delete from ordenes where id = $1', [o.id]);
    return { ref: o.ref, eliminada: true };
  });
}

const TEL_MAX = 30;

function validarCantidades(entradas, sueltos) {
  if (!Number.isInteger(entradas) || !Number.isInteger(sueltos) || entradas < 0 |
    throw new ErrorNegocio('cantidad_invalida', 'Revisá la cantidad de entradas y
  }
  if (entradas + sueltos < 1 || entradas + sueltos > MAX_POR_ORDEN) {
    throw new ErrorNegocio('cantidad_invalida', `Tiene que haber entre 1 y ${MAX_
  }
}

/**
 * Corrige a mano las cantidades de una compra ya pagada (o de una cortesía): sum
 * sueltos. Los cartones que sobran vuelven al stock y los que faltan se toman al
 * devuelve `diferencia` (negativa = hay que devolver a la persona en Mercado Pag
 */
export async function ajustarCompra(pool, { ref, entradas, sueltos }) {
  validarCantidades(entradas, sueltos);
  return tx(pool, async (c) => {
    const o = (await c.query('select * from ordenes where ref = $1 for update', [
    if (!o) throw new ErrorNegocio('orden_inexistente', 'No encontramos esa compr
    if (o.estado !== 'pagada') throw new ErrorNegocio('no_ajustable', 'Solo se pu
    const antes = o.cantidad, despues = entradas + sueltos;
    if (despues < antes) {
      const sobran = (await c.query(
        `select numero from numeros where orden_id = $1 and estado = 'vendido' or
        [o.id, antes - despues])).rows.map((r) => r.numero);
      await c.query(
        `update numeros set estado = 'libre', orden_id = null, reservado_hasta =
    } else if (despues > antes) {
      const nuevos = await tomarAzar(c, despues - antes);
```

```
      if (nuevos.length !== despues - antes) throw new ErrorNegocio('sin_stock',
      await c.query(
        `update numeros set estado = 'vendido', orden_id = $1, reservado_hasta =
    }
    const pagado = o.cortesia ? 0 : (o.monto_original ?? o.monto);
    const monto = o.cortesia ? 0 : montoDe(entradas, sueltos);
    await c.query(
      `update ordenes set entradas = $2, cantidad = $3, monto = $4, monto_origina
      [o.id, entradas, despues, monto, o.cortesia ? null : pagado]);
    return { ref: o.ref, entradas, sueltos, cantidad: despues, monto, pagado, dif
  });
}

/**
 * Pasa entradas y cartones sueltos de una compra a otra (por ejemplo, alguien le
 * Los cartones cambian de dueño, no se vuelven a sortear. Si las dos compras son
 * ellos, así que ninguna de las dos queda con "devolver" ni "falta cobrar" (y la
 * Si una es cortesía, no se mueve plata.
 */
export async function transferirCompra(pool, { desde, hacia, entradas, sueltos })
  if (!Number.isInteger(entradas) || !Number.isInteger(sueltos) || entradas < 0 |
    throw new ErrorNegocio('cantidad_invalida', 'Indicá cuántas entradas y cuánto
  }
  const refA = String(desde ?? ''), refB = String(hacia ?? '');
  if (!refA || !refB || refA === refB) throw new ErrorNegocio('compras_invalidas'
  const k = entradas + sueltos;
  return tx(pool, async (c) => {
    const filas = (await c.query('select * from ordenes where ref = any($1) order
    const A = filas.find((o) => o.ref === refA), B = filas.find((o) => o.ref ===
    if (!A || !B) throw new ErrorNegocio('orden_inexistente', 'No encontramos una
    if (A.estado !== 'pagada' || B.estado !== 'pagada') throw new ErrorNegocio('n
    if (A.entradas < entradas || (A.cantidad - A.entradas) < sueltos) {
      throw new ErrorNegocio('cantidad_invalida', 'La compra de origen no tiene t
    }
    if (A.cantidad - k < 1) throw new ErrorNegocio('queda_vacia', 'La compra de o
    if (B.cantidad + k > MAX_POR_ORDEN) throw new ErrorNegocio('cantidad_invalida
    const mover = (await c.query(
      `select numero from numeros where orden_id = $1 and estado = 'vendido' orde
    await c.query(`update numeros set orden_id = $1 where numero = any($2::smalli
    const mueveMonto = !A.cortesia && !B.cortesia;
    const v = mueveMonto ? montoDe(entradas, sueltos) : 0;
    const pagadoA = A.monto_original ?? A.monto, pagadoB = B.monto_original ?? B.
    await c.query(
      `update ordenes set entradas = $2, cantidad = $3, monto = $4, monto_origina
      [A.id, A.entradas - entradas, A.cantidad - k, A.monto - v, mueveMonto ? pag
    await c.query(
      `update ordenes set entradas = $2, cantidad = $3, monto = $4, monto_origina
```

```
      [B.id, B.entradas + entradas, B.cantidad + k, B.monto + v, mueveMonto ? pag
    return { desde: A.ref, hacia: B.ref, de: A.comprador_nombre, a: B.comprador_n
  });
}

/** Regala entradas y cartones (invitación sin pago): queda en el registro como "
export async function crearCortesia(pool, { nombre, whatsapp, entradas, sueltos }
  validarCantidades(entradas, sueltos);
  if (!nombreCompletoValido(nombre)) throw new ErrorNegocio('nombre_invalido', 'E
  const tel = String(whatsapp ?? '').trim().slice(0, TEL_MAX) || 'sin teléfono';
  const cantidad = entradas + sueltos;
  return tx(pool, async (c) => {
    const id = randomUUID(), ref = nuevaRef();
    const numeros = await tomarAzar(c, cantidad);
    if (numeros.length !== cantidad) throw new ErrorNegocio('sin_stock', 'No qued
    await c.query(
      `insert into ordenes (id, ref, comprador_nombre, comprador_whatsapp, cantid
                            cortesia, expira_en, pagada_en)
       values ($1, $2, $3, $4, $5, $6, 'azar', 0, 'pagada', true, now(), now())`,
      [id, ref, formatearNombre(nombre), tel, cantidad, entradas]);
    await c.query(
      `update numeros set estado = 'vendido', orden_id = $1, reservado_hasta = nu
    return { ref, comprador: formatearNombre(nombre), entradas, sueltos, cantidad
  });
}

// Cruza órdenes, pagos y números. Todo en `problemas` debe ser cero.
export async function conciliacion(pool) {
  const q = (sql) => pool.query(sql).then((r) => r.rows);
  const vendidos = (await q(`select count(*)::int n from numeros where estado = '
  const esperados = (await q(`select coalesce(sum(cantidad),0)::int n from ordene
  const reembolsos = await q(
    `select mp_payment_id, external_reference, estado_mp, monto, resultado, detal
     where resultado in ('requiere_reembolso','sin_orden','monto_distinto')
        or (resultado = 'asignado' and estado_mp <> 'approved')`);
  const pagadasSinPago = await q(
    `select o.ref from ordenes o where o.estado = 'pagada' and not o.cortesia
       and not exists (select 1 from pagos p where p.orden_id = o.id and p.result
  const vendidosSinOrden = await q(
    `select n.numero from numeros n left join ordenes o on o.id = n.orden_id
     where n.estado = 'vendido' and (o.id is null or o.estado <> 'pagada')`);
  const pagadasSinNumeros = await q(
    `select o.ref from ordenes o where o.estado = 'pagada'
       and (select count(*) from numeros n where n.orden_id = o.id and n.estado =
  const canceladasConPago = await q(
    `select o.ref, p.mp_payment_id, p.monto from ordenes o join pagos p on p.orde
     where o.estado = 'cancelada' and p.estado_mp = 'approved'`);
```

```
  return {
    coherente: vendidos === esperados && !pagadasSinPago.length && !vendidosSinOr
    vendidos, esperados,
    problemas: { reembolsos, pagadasSinPago, vendidosSinOrden, pagadasSinNumeros
    reintegrosPendientes: canceladasConPago,
  };
}
```